

Side Channel Experiment : Differential Power Analysis

Table of Content

Side Channel Experiment : Differential Power Analysis.....	1
1. Introduction	3
2. Background	4
2.1 Power trace	4
2.2 Differential power analysis: high-level concept with AES (10 rounds – 256 bits key)	4
3. Getting Started.....	6
3.1 Downloading and Installing python libraries	6
3.2 Python tips and commands.....	7
4. Lab Instructions	8
5. Method	9
6. References	13

1. Introduction

Differential Power Analysis (DPA) is a type of attack which leverages the fluctuations in the power consumption of the device to make data-dependent inferences. Indeed, every time a transistor inside a chip changes its state (from 0 to 1 or vice-versa), the power consumption varies.

Therefore, when an encryption process takes place, the device's overall power consumption will fluctuate over time depending on the actual data and the operations being processed (the encryption algorithm).

In this lab, we will conduct a DPA on AES (10 rounds – 128-bits key). AES (Advanced Encryption Standard) is a symmetric encryption algorithm that comes in three key lengths: 128, 192, and 256 bits. The longer the key, the more secure but also slightly slower the encryption. Each key length determines the number of rounds the data goes through during encryption: 10 rounds for AES-128, 12 for AES-192, and 14 for AES-256 [2].

Plaintexts, ciphertexts and their related power traces are provided. In short, to perform the DPA, we will:

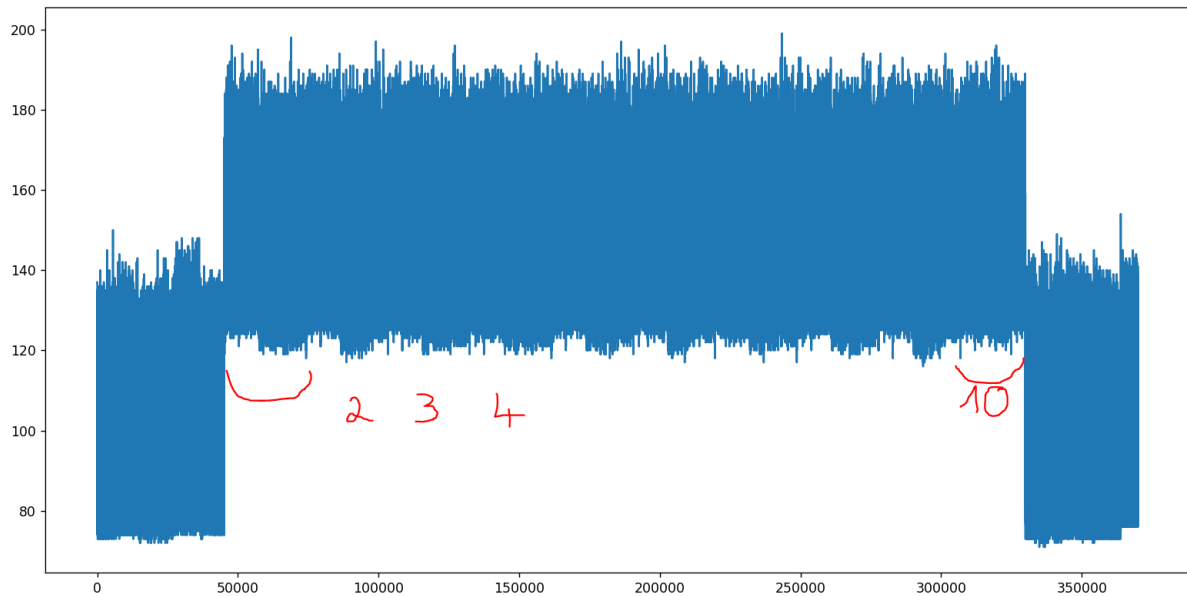
- Brute-force the key, byte by byte, by performing the AES encryption with the known plaintexts and all the 256 possible permutations of the 8-bit partial key.
We only simulate the encryption up to an intermediate value, somewhere in the 10 rounds, that is 1) linked to both the plaintext and the key (and the power consumption levels), and 2) a lot easier/faster to compute than the full 10 rounds.
- Apply a power model to the obtained intermediate values to express their “power consumption profile”.
- Compare (correlate) all those brute-forced power values with the real measured ones (power traces) to reveal the right key.
- Repeat this process for all 16 bytes of the plaintext/key.

This process will be broken down in more practical details in section 5 of this document.

2. Background

2.1 Power trace

A power trace is a succession of power consumption levels over time. In this lab, they have been captured during the encryption process of AES. Hereunder is an example of such power trace. In this case, we can clearly see the power consumption fluctuations for the 10 AES rounds:



2.2 Differential power analysis: high-level concept with AES (10 rounds – 128 bits key)

In AES, even when provided with both the plaintext and cipher text, modern computers are not powerful enough to extrapolate the key fast enough. This is where DPA comes in.

IoT and other portable devices usually lack the memory capacity and computing power to store multiple (or generate new) encryption keys. This is why those devices usually have 1 hard-coded or “burnt in the chip” unique encryption key. The same key is thus used for all encryptions.

To make the key cracking process more manageable, and given how AES is implemented, we will compute 1 byte of the plaintext at a time to reconstruct the corresponding byte of the key. The process needs to be repeated for every byte of the plaintext to recover the full key.

The concept of the attack is as follows:

- 1) We can use the given plaintext and encrypt it with a key guess, up until the chosen intermediary state to obtain an intermediary cipher text.
We proceed with this step with all 256 possible 1-byte sized keys to effectively perform a brute-force attack of the partial key, and obtain as many intermediate values.
- 2) In parallel we pick a power model function which, applied to our intermediate values, will provide an accurate enough representation of the device’s power consumption for that value.

Examples of such power functions are the Hamming Weight (HW) and the Hamming Distance (HD). As a reminder, the HW counts the number of bits set to 1 in the binary representation of a given value.

The more 1's in a given value, the more energy the device will need just to store and process that value. HW is thus indeed a value that influences the power consumption of the device to some extent.

- 3) Finally, we correlate the real power values (measured during the actual encryption process) with the hypothetical ones. The hypothetical value yielding the highest correlation thus is the real key. In real-life context, the attacker has to collect the power traces by measuring the target device's consumption during the encryption process. In this lab, for practical purposes, those traces are provided.

On one hand, the Hamming distance will only provide an approximation of the real power consumption of the device. On the other hand, noise can alter the measurements made to obtain the "real" consumption values. Therefore, the right key guess might not elicit the highest correlation in power consumption with the real power traces. To overcome this, we need to collect a lot of power traces. Indeed, the more traces, the more we'll be able to statistically reduce the impact of those interferences and imprecisions.

The fact that each trace uses a different plaintext doesn't matter since all of them use the same encryption key. Therefore, the same key should elicit the highest correlation between hypothetical and real power values for all the traces.

3. Getting Started

In this lab, we will need to use a program suitable for numerical calculations. Python, using the right libraries (matplotlib and numpy), is a suitable option.

3.1 Downloading and Installing python libraries

Python is one of the most popular programming languages today, praised for simple and intuitive syntax and easy to use libraries.

The python code for this lab is written using python 3. What you need to run the code is:

- python 3.X. Code has been tested with version 3.7.
- matplotlib. Library containing pyplot, which is used to plot graphs.
- numpy. Library for better linear algebra and vector operations.
- (optional) jupyter. Used to run jupyter notebook to open the .ipynb file.
- (very optional) qt. Used for window/GUI management.

Depending on factors like your operating system this will be done differently. However, it is recommended that you install conda for python environment management. If you choose to use conda then the process is the same regardless of operating system. The instructions for how to install conda for your operating system can be found at:

<https://docs.conda.io/projects/conda/en/latest/user-guide/install/>

Assuming you chose to install conda, the dependencies can be installed like this:

1. Open a terminal with conda installed. On Linux this will be a regular bash terminal. On Windows you should have a new program called anaconda after installation which opens a CLI with conda.
2. (optional) Create a new conda environment using the command:

```
conda create -n labenv python=3.7
```

then activate it using:

```
conda activate labenv
```

3. Install libraries using these commands:

```
conda install matplotlib
```

```
conda install numpy
```

```
conda install jupyter
```

If you installed jupyter through conda then you already get part of the qt framework library included.

4. Open the code in jupyter notebook using the command:

```
jupyter notebook CPA.ipynb
```

then run the first code block. If you get no errors then it means all libraries have been installed correctly.

3.2 Python tips and commands

For this section, please refer to the python code. There's a tutorial with all the tips and commands you need to finish the lab at the beginning of the code.

If you, for some reason, need to know more than what is shown in the code then you can refer to the online documentation. There will be links to the numpy documentation and pyplot documentation below, but it is faster to use google. E.g. if you search for "numpy covariance" or "pyplot scatter" then the top results are usually links to the official documentation.

Numpy documentation: <https://numpy.org/doc/>

Pyplot documentation: https://matplotlib.org/3.3.4/api/_as_gen/matplotlib.pyplot.html

4. Lab Instructions

- 1) Carefully read the next section to understand the practicalities of the attack.
- 2) Go through the Python tutorial within the *python-tutorial.py* file to learn basic commands.
- 3) Write the code to perform the attack within the DPA-template.py file (it contains premade functions for the more complex steps of the DPA that you don't need to code yourself).
- 4) Verify your code by retrieving the known "00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff" key with the related files:

- plaintext-00112233445566778899aabbccddeeff.txt
- traces-00112233445566778899aabbccddeeff.npy

- 5) Use your code to retrieve the unknown key with the related files:

- plaintext-unknown_key.txt
- traces-unknown_key.npy

You can verify that the retrieved key is the right one by encrypting a

- 6) Write a report answering the following questions and upload it in pdf format on Canvas (named "Group_X_DPA.pdf", where X is your group number):
 - a. Submit the 128-bit AES unknown key in hexadecimal format, no spaces.
 - b. Did you use a custom time/sample range to find that key? If yes, which one and why?
 - c. Plot the last trace in full-screen window size and provide a screenshot of it.
 - d. At what sample index/time offset did the strongest correlation for byte 0 appear?
 - e. In your own words, briefly explain why does correlating traces against the HW of the first round S-box output typically produce a single clear peak at the correct key-byte guess, whereas the wrong guesses show only low-level noise? Outline the key statistical reason.

5. Method

Here are the practical steps explained in further details.

1. Run encryption on the device and measure its power consumption
 - a. This had been done. The power traces and their related plaintexts are provided.

There are 200 power traces, each containing 370.000 samples. Each sample is represented by an 8-bit unsigned value.

Those traces and samples are represented in a matrix with the rows being the traces, and the columns, their related samples for each time unit. Below, a visual example for 500 traces of 50000 samples each:

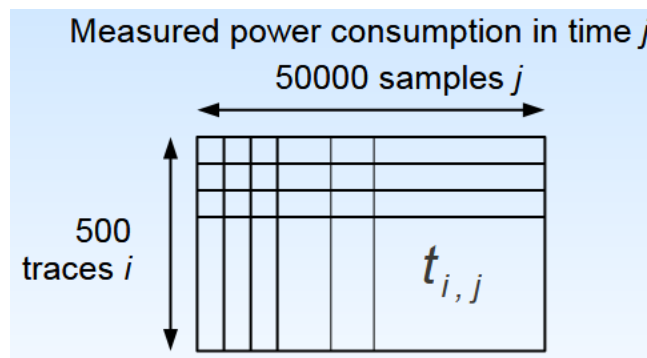


Figure 1 adapted from Buček, J., & Novotný, M. (2014, July) [1]

2. Pick an intermediate value of the encryption process
 - a. “The element of AES that correlates to the power consumption is the Hamming weight of the output of the SBOX operation after the very first SubBytes step” [3]. The transient cipher output of the said first substitution of the first round is thus a good candidate for the intermediate value.

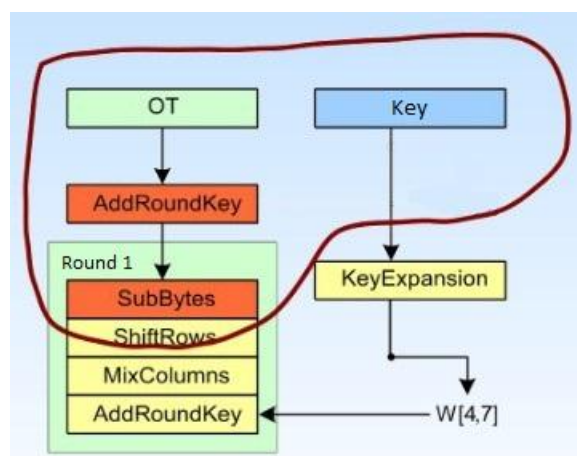


Figure 2 adapted from Buček, J., & Novotný, M. (2014, July) [1]

3. Make a key guess

- a. For this attack to work, all keys must be tried. A key “guess” will thus translate into a brute-force of all 256 possible values for the key byte.

All 256 possible keys are brute-forced for 1 trace. Let’s proceed with each trace. This gives us a matrix where the columns are all possible keys, with as many rows as there are traces. Below, a visual representation of such matrix, continuing with our previous example of 500 traces with 50000 samples each:

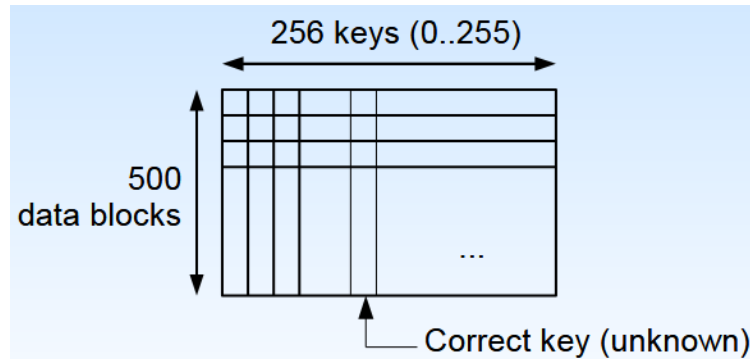


Figure 3 adapted from Buček, J., & Novotný, M. (2014, July) [1]

4. Compute the hypothetical intermediate value

- a. With the known plaintext (a) and the guessed key (k) from the brute-force, we compute the resulting intermediate value (v), as if we were performing the AES encryption normally. We just stop after the first substitution (S operation).

For all i’s (from 0 to 15), where i identifies the byte from the plaintext we’re currently addressing, and k’s (from 0 to 255), we thus have:

$$v_{i,k} = S(a_i \oplus k)$$

So, with our previous key matrix (where each cell is a possible key), we replace the said cells with their resulting partial encryption until the intermediate value. From the matrix of all key for each trace, we obtain the matrix of all intermediate values for each trace.

Note that each trace is a different encryption process with a different plaintext from the others (but all the same key).

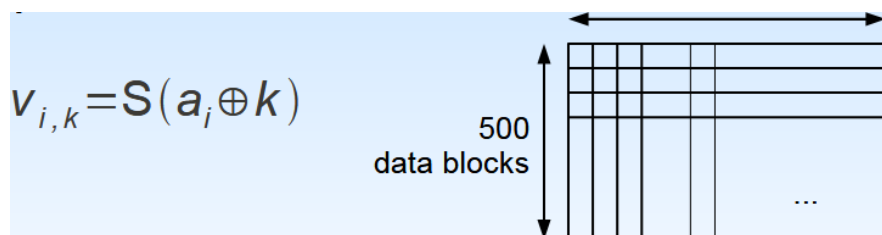


Figure 4 adapted from Buček, J., & Novotný, M. (2014, July) [1]

5. Select and apply a power model on the hypothetical value

- a. The Hamming Weight (HW) is a good candidate for a power model, as its value is directly linked the power consumption of the device. We thus have:

$$H_{i,k} = HW(S(a_i \oplus k))$$

From the matrix of all intermediate values, we obtain the matrix of their hypothetical consumption values:



Figure 5 adapted from Buček, J., & Novotný, M. (2014, July) [1]

6. Correlate the result with the power traces

- a. Run the Pearson's correlation between the hypothetical power values and the actual values from the measured power traces. The highest correlation value reveals the right key guess. A premade Python function is provided.

What the Pearson's correlation does is to determine if, when the measured power traces fluctuate, do the hypothetical power consumption values vary the same way?

The Pearson's correlation outputs a single value closer to -1 if pair values are inversely correlated, closer to 1 if they are positively correlated, and closer to 0 if they are not correlated.

Out of all the guessed keys, leading to all the hypothetically possible power consumption values, the one that is the most correlated with the actual power traces is thus the one with the right encryption key!

- b. But in our example, there are 50000 traces (columns) for the real values, and only 256 possible keys (columns) for the hypothetical ones.

It is crucial to understand that our hypothetical values are tied to 1 specific time point of the AES encryption process. However, we don't know to which one (time t) it corresponds in the 50000 samples. Therefore, we are forced to test them all.

We will thus proceed with a correlation calculation on a column basis, going through all t samples of the real values.

Concretely, we will start with the first key guess (1st column of hypothetical values) and correlate it with:

- the 1st column of the real values ($t = 0$): obtain a correlation value
- the 2nd column of the real values ($t = 1$): obtain a correlation value
- the 3rd column of the real values ($t = 2$): obtain a correlation value
- ...

Once all 50000 columns have been tested, we start over with the second key guess (second column of hypothetical power consumption matrix), etc, until we processed all 256 key guesses.

The highest correlation value of all those computations will reveal the right key!

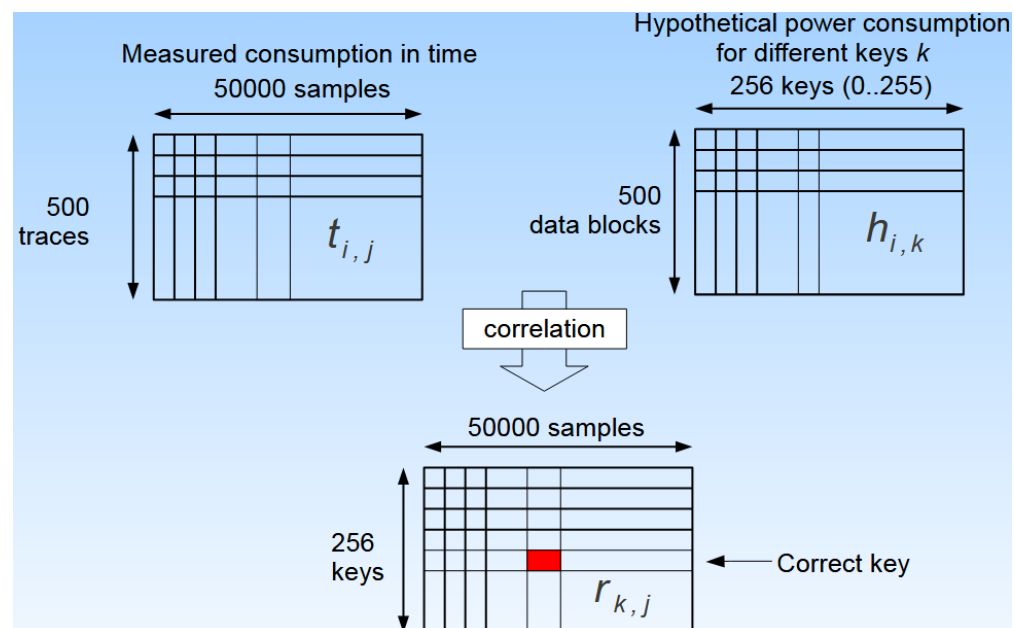


Figure 6 adapted from Buček, J., & Novotný, M. (2014, July) [1]

Note: To drastically reduce the computational load, it is therefore interesting to reduce the scope of the real values to test. Indeed, since our intermediate value of interest is in the 1st round of AES, there is no benefit in testing our hypothetical values on the samples of the rounds 2 to 10.

We can visually (roughly) isolate the corresponding t range of the 1st round by plotting the full power trace. This is the 1st step of the resolution guide from the Python tutorial file.

6. References

- [1] Buček, J., & Novotný, M. (2014, July). *Introduction to differential power analysis with correlation coefficients* [PowerPoint slides]. Training School on Trustworthy Manufacturing and Utilization of Secure Devices (TRUDEVICE 2014), Lisbon, Portugal. CTU in Prague, Faculty of Information Technology.
- [2] GeeksforGeeks. (2025, July 23). *Advanced Encryption Standard (AES)*. GeeksforGeeks. <https://www.geeksforgeeks.org/computer-networks/advanced-encryption-standard-aes/>
- [3] Lathrop, L. (2020). Differential power analysis attacks on different implementations of AES with the ChipWhisperer Nano. *Cryptology ePrint Archive*.